

Object Oriented Programming

Lecture#6 Inheritance

การสืบทอด (Inheritance)

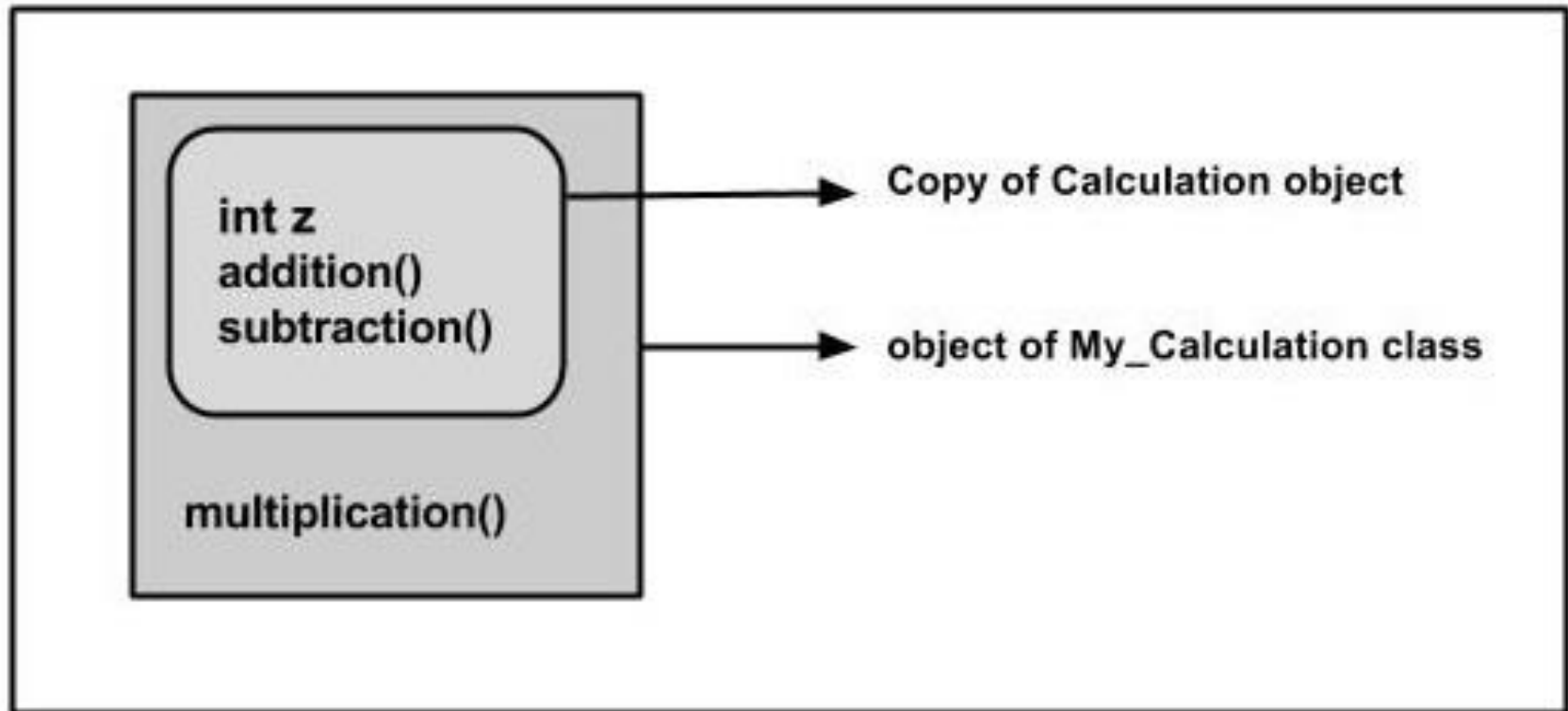
- ▶ การเขียนโปรแกรมเชิงวัตถุ จะอนุญาตให้สามารถสร้างคลาสใหม่จากคลาสเดิมที่เคยมีอยู่แล้วซึ่งเรียกเทคนิคนี้ว่าการสืบทอด (Inheritance)
 - ▶ การสืบทอด(Inheritance) เป็นเทคนิคที่สำคัญมากสำหรับการนำซอฟต์แวร์กลับมาใช้ใหม่ (Reuse software)
 - ▶ สมมติต้องการคลาสวงกลม สีเหลี่ยมและสามเหลี่ยม คลาสต่าง ๆ เหล่านี้จะมีคุณลักษณะบางอย่างที่คล้ายกัน การออกแบบคลาสเพื่อไม่ให้ใช้ตัวแปรซ้ำ และทำให้ง่ายต่อการแก้ไข วิธีที่ดีที่สุดคือการใช้หลักการของการสืบทอดข้อมูล
- ▶ Super class และ Sub class
 - ▶ เทคนิคการสืบทอดข้อมูลอนุญาตให้กำหนดคลาสทั่วไปที่เรียกว่า General class หรือ Super class และภายหลังสามารถขยาย (extends) คลาสดังกล่าวไปเป็นคลาสที่มีความจำเพาะเจาะจงได้ specialized class หรือ sub class
 - ▶ เราสามารถใช้คลาสเพื่อจำลองวัตถุที่มีชนิดข้อมูลเดียวกันได้
 - ▶ คลาสต่าง ๆ กันอาจจะมีคุณสมบัติและพฤติกรรมที่เหมือนกัน ที่สามารถทำให้อยู่ในรูปทั่วไปได้และสามารถใช้ร่วมกับคลาสอื่น ๆ ได้
 - ▶ เรายังสามารถกำหนดคลาสที่มีคุณสมบัติจำเพาะ ที่สืบทอดหรือขยายมาจากคลาสทั่วไปหรือ Generalized class ได้โดยที่คลาสที่มีความจำเพาะเหล่านั้นสืบทอดคุณสมบัติและเมธอดมาจากคลาสทั่วไป

การสืบทอด (Inheritance)

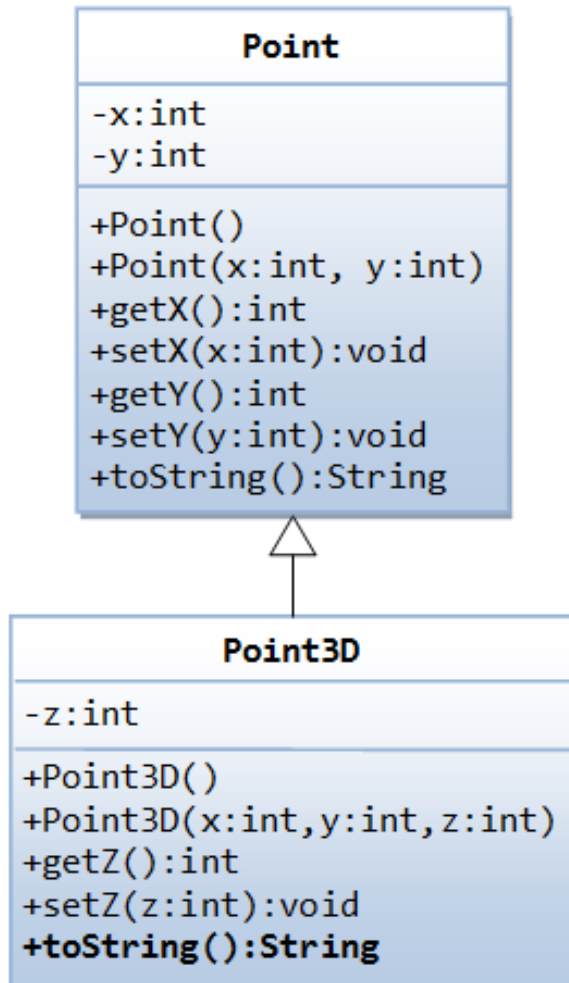
```
► class Calculation{
    int z;
    public void addition(int x, int y){
        z=x+y;
        System.out.println("The sum of the given numbers:"+z);
    }
    public void Substraction(int x,int y){
        z=x-y;
        System.out.println("The difference between the given numbers:"+z);
    }
}

public class My_Calculation extends Calculation{
    public void multiplication(int x, int y){
        z=x*y;
        System.out.println("The product of the given numbers:"+z);
    }
    public static void main(String args[]){
        int a=20, b=10;
        My_Calculation demo = new My_Calculation();
        demo.addition(a, b);
        demo.Substraction(a, b);
        demo.multiplication(a, b);
    }
}
```

การสืบทอด (Inheritance)

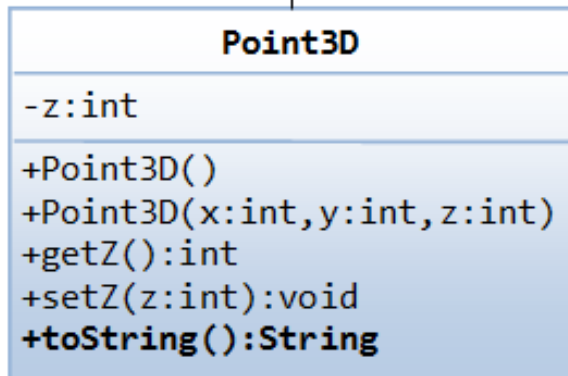
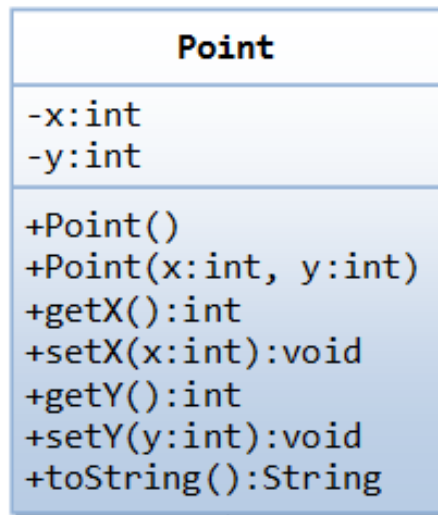


การสืบทอด (Inheritance)



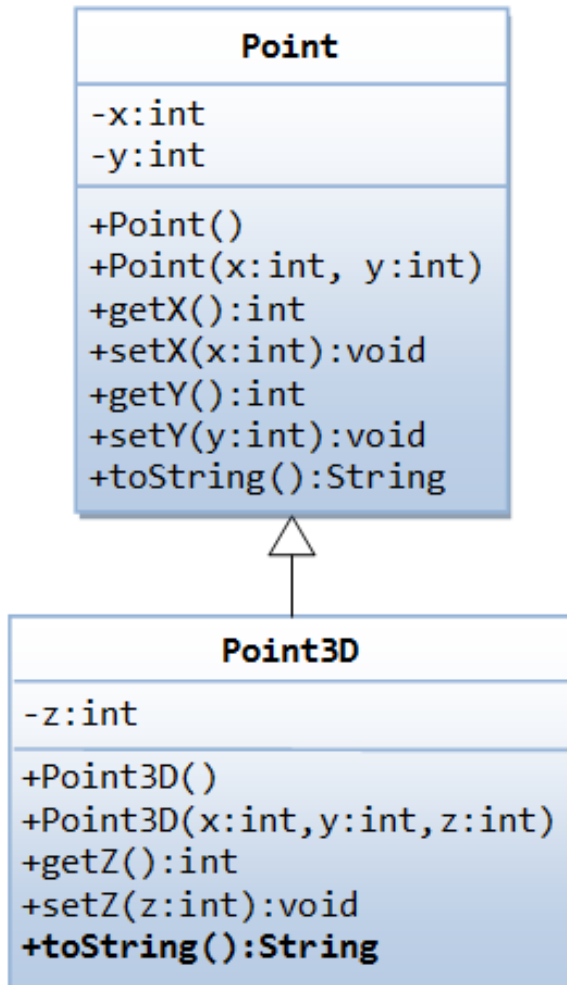
```
// The Point class definition
public class Point {
    // Private member variables
    private int x, y; // (x, y) co-ordinates
    // Constructors
    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }
    public Point() { // default (no-arg) constructor
        x = 0;
        y = 0;
    }
    // Public getter and setter for private variables
    public int getX() {
        return x;
    }
    public void setX(int x) {
        this.x = x;
    }
    public int getY() {
        return y;
    }
    public void setY(int y) {
        this.y = y;
    }
    // toString() to describe itself
    public String toString() {
        return "(" + x + "," + y + ")";
    }
}
```

การสืบทอด (Inheritance)



```
// Define Point3D, subclass of Point
public class Point3D extends Point {
    // Private member variable
    private int z;
    // Constructors
    public Point3D() { // default no-arg constructor
        super();      // Call superclass' no-arg constructor Point()
        z = 0;
    }
    public Point3D(int x, int y, int z) {
        super(x, y);  // Call superclass' Point(x, y)
        this.z = z;
    }
    // Public getter/setter for private variable
    public int getZ() {
        return z;
    }
    public void setZ(int z) {
        this.z = z;
    }
    // toString() to describe itself
    @Override
    public String toString() {
        return "(" + super.getX() + "," + super.getY() + "," + z + ")";
    }
}
```

การสืบทอด (Inheritance)



```
// A test driver program for Point3D class
public class TestPoint3D {
    public static void main(String[] args) {
        Point3D p1 = new Point3D(1, 2, 3);
        System.out.println(p1);
        System.out.println(p1.getX());
        // Inherited from superclass
        System.out.println(p1.getY());
        // Inherited from superclass
        System.out.println(p1.getZ());
        // this class
        p1.setX(4);
        // Inherited from superclass
        p1.setY(5);
        // Inherited from superclass
        p1.setZ(6);
        // this class
        System.out.println(p1);
    }
}
```

Superclasses and Subclasses

GeometricObject
-color: String -filled: boolean -dateCreated: java.util.Date
+GeometricObject() +GeometricObject(color: String, filled: boolean) +getColor(): String +setColor(color: String): void +isFilled(): boolean +setFilled(filled: boolean): void +getDateCreated(): java.util.Date +toString(): String

The color of the object (default: white).

Indicates whether the object is filled with a color (default: false).

The date when the object was created.

Creates a GeometricObject.

Creates a GeometricObject with the specified color and filled values.

Returns the color.

Sets a new color.

Returns the filled property.

Sets a new filled property.

Returns the dateCreated.

Returns a string representation of this object.

Circle
-radius: double
+Circle() +Circle(radius: double) +Circle(radius: double, color: String, filled: boolean) +getRadius(): double +setRadius(radius: double): void +getArea(): double +getPerimeter(): double +getDiameter(): double +printCircle(): void

Rectangle
-width: double -height: double
+Rectangle() +Rectangle(width: double, height: double) +Rectangle(width: double, height: double, color: String, filled: boolean) +getWidth(): double +setWidth(width: double): void +getHeight(): double +setHeight(height: double): void +getArea(): double +getPerimeter(): double



Are superclass's Constructor Inherited?

No. They are not inherited.

They are invoked explicitly or implicitly.

Explicitly using the `super` keyword.

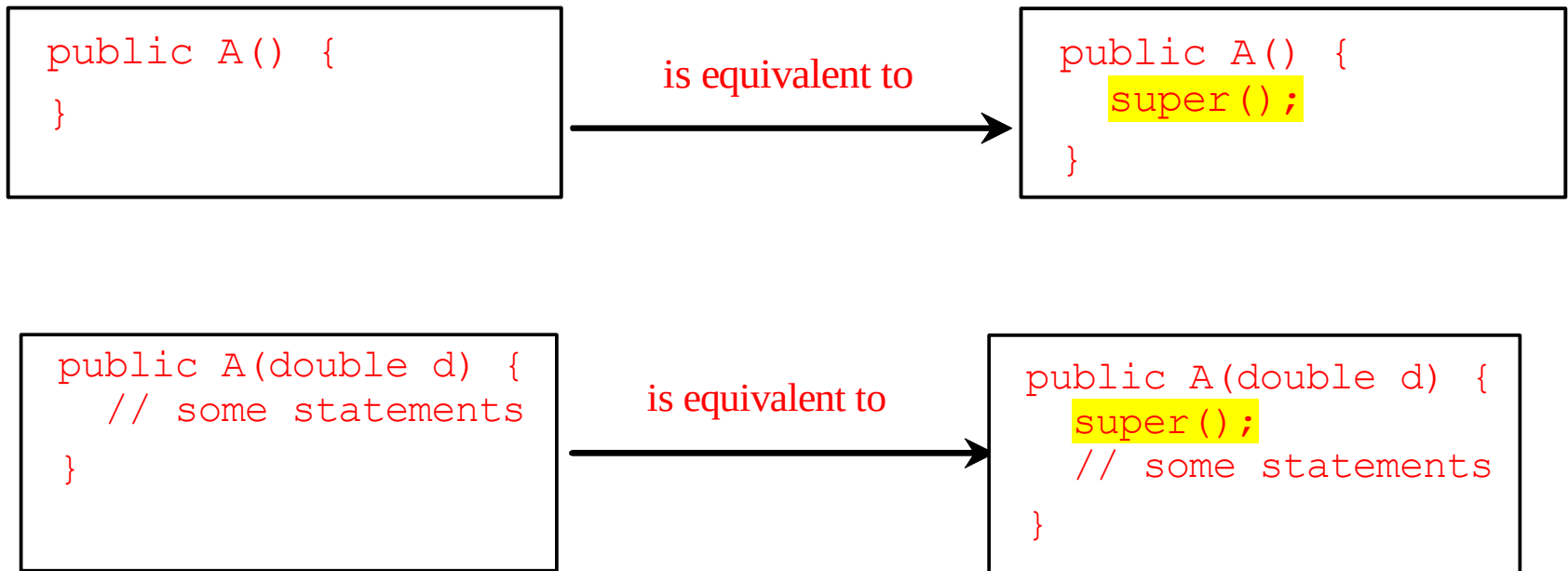
- ❑ A **constructor** is used to construct an instance of a class.
- ❑ Unlike properties and methods, a superclass's constructors are not inherited in the subclass.
- ❑ They can only be invoked from the subclasses' constructors, using the keyword super.
- ❑ *If the keyword super is not explicitly used, the superclass's no-arg constructor is automatically invoked.*

การใช้คีย์เวิร์ด **super**

- ▶ คีย์เวิร์ด `super` ใช้อ้างถึง `super class` และสามารถใช้สำหรับเรียกเมธอดและคอนสตรักเตอร์ของ `super class` ได้ โดยที่ `subclass` สามารถสืบทอดข้อมูลและเมธอดที่สามารถเข้าถึงได้ จาก `super class` เราสามารถใช้ `super` ในการอ้างอิงโดยมีรูปแบบการใช้งานหลัก ๆ 2 รูปแบบคือ
 - ▶ ใช้เรียกคอนสตรักเตอร์ของ `super class`
 - ▶ ใช้เรียกเมธอดของ `super class`
- ▶ คอนสตรักเตอร์ (Constructor) ใช้สำหรับสร้าง instance หรือวัตถุของคลาส คอนสตรักเตอร์ของ `super class` จะไม่ถูกสืบทอดโดย `subclass` แต่สามารถเรียกคอนสตรักเตอร์ของ `superclass` จาก `subclass` โดยการใช้ `super` โดยรูปแบบในการเรียกใช้คอนสตรักเตอร์ของ `super class` คือ`super()` หรือ `super(parameter)`

Superclass's Constructor Is Always Invoked

A constructor may invoke an overloaded constructor or its superclass's constructor. If none of them is invoked explicitly, the compiler puts super() as the first statement in the constructor. For example,



ข้อควรระวัง

การเรียกใช้คอนสตรัคเตอร์ของ superclass จะใช้คีย์เวิร์ด super

การเรียกใช้คอนสตรัคเตอร์ของ superclass ด้วยการระบุชื่อของ constructor จะทำให้เกิด syntax error.

Constructor Chaining

ในขณะที่กำลังสร้าง Instance ของคลาสจะมีการเรียกคอนสตรัคเตอร์ของ superclass ทั้งหมดที่อยู่ในโครงสร้างการสืบทอดเดียวกันเราเรียกลักษณะการทำงานรูปแบบนี้ว่า constructor chaining

```
public class Faculty extends Employee {
    public static void main(String[] args) {
        new Faculty();
    }
    public Faculty() {
        System.out.println("(4) Faculty's no-arg constructor is invoked");
    }
}
class Employee extends Person {
    public Employee() {
        this("(2) Invoke Employee's overloaded constructor");
        System.out.println("(3) Employee's no-arg constructor is invoked");
    }
    public Employee(String s) {
        System.out.println(s);
    }
}

class Person {
    public Person() {
        System.out.println("(1) Person's no-arg constructor is invoked");
    }
}
```

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

1. Start from the
main method

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
}
```

2. Invoke Faculty
constructor

```
public Faculty() {  
    System.out.println("(4) Faculty's no-arg constructor is invoked");  
}  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

3. Invoke Employee's no-arg constructor

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
}
```

```
public Faculty() {  
    System.out.println("(4) Faculty's no-arg constructor is invoked");  
}  
}
```

**4. Invoke Employee(String)
constructor**

```
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
}
```

```
public Employee(String s) {  
    System.out.println(s);  
}  
}
```

```
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

5. Invoke Person() constructor

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

6. Execute println

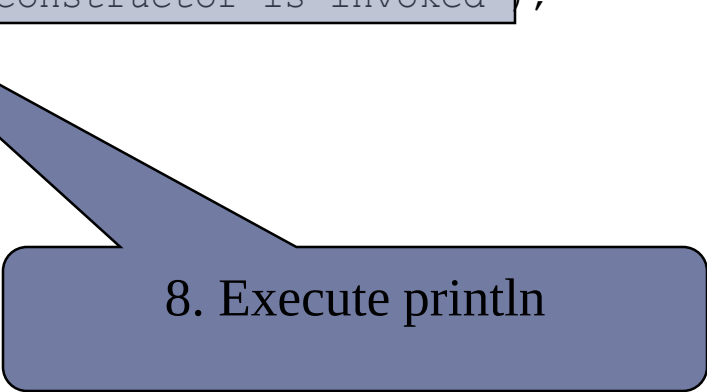
Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```

7. Execute println

Trace Execution

```
public class Faculty extends Employee {  
    public static void main(String[] args) {  
        new Faculty();  
    }  
  
    public Faculty() {  
        System.out.println("(4) Faculty's no-arg constructor is invoked");  
    }  
}  
  
class Employee extends Person {  
    public Employee() {  
        this("(2) Invoke Employee's overloaded constructor");  
        System.out.println("(3) Employee's no-arg constructor is invoked");  
    }  
  
    public Employee(String s) {  
        System.out.println(s);  
    }  
}  
  
class Person {  
    public Person() {  
        System.out.println("(1) Person's no-arg constructor is invoked");  
    }  
}
```



8. Execute println

Trace Execution

```
public class Faculty extends Employee {
    public static void main(String[] args) {
        new Faculty();
    }

    public Faculty() {
        System.out.println("(4) Faculty's no-arg constructor is invoked");
    }
}

class Employee extends Person {
    public Employee() {
        this("(2) Invoke Employee's overloaded constructor");
        System.out.println("(3) Employee's no-arg constructor is invoked");
    }

    public Employee(String s) {
        System.out.println(s);
    }
}

class Person {
    public Person() {
        System.out.println("(1) Person's no-arg constructor is invoked");
    }
}
```

9. Execute println

Example on the Impact of a Superclass without no-arg Constructor

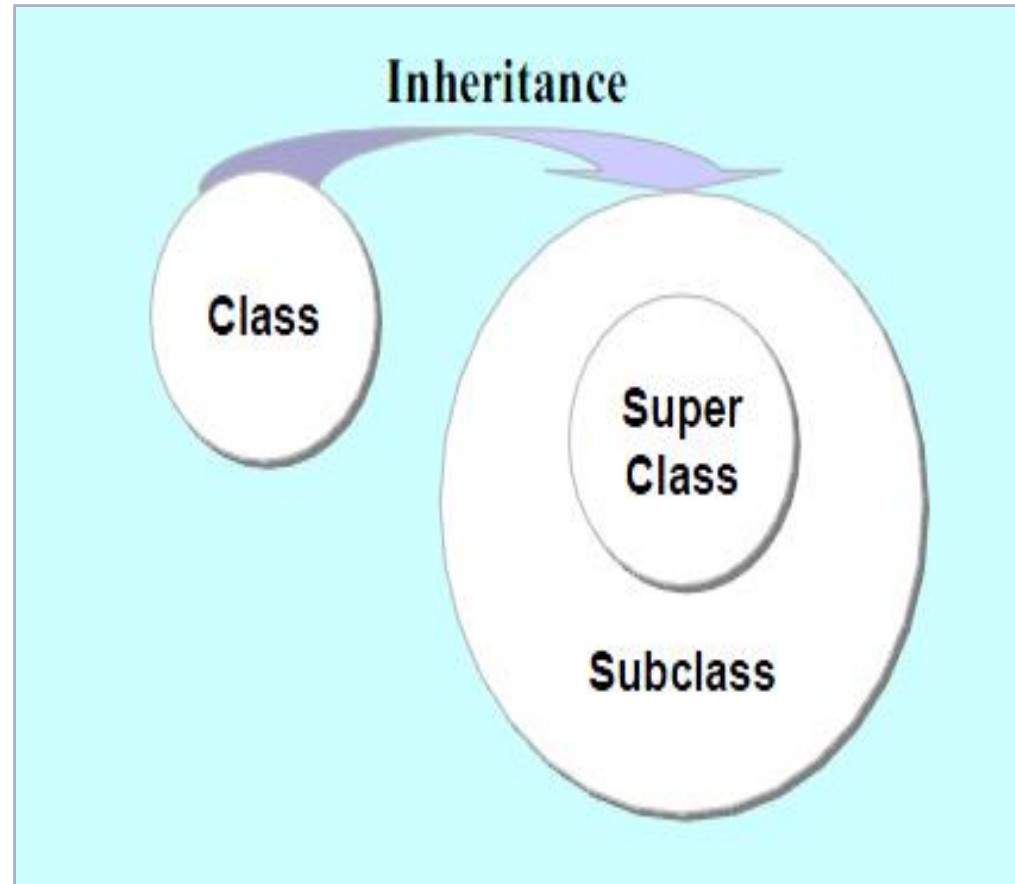
Find out the errors in the program:

```
public class Apple extends Fruit {  
}  
  
class Fruit {  
    public Fruit(String name) {  
        System.out.println("Fruit's constructor is invoked");  
    }  
}
```

การประกาศ Subclass (Declaring a Subclass)

□ subclass สามารถเพิ่มคุณสมบัติและเมธอดจาก superclass โดยสามารถเพิ่มสิ่งเหล่านี้

- new properties
- new methods
- Override the methods of the superclass



Calling Superclass Methods

ตัวอย่างการเรียกเมธอดของคลาสแม่ ตัวอย่างต้องการเขียนเมธอด printCircle() ในคลาส Circle ใหม่จะสามารถเขียนได้ดังนี้

```
public void printCircle() {  
    System.out.println("The circle is created " +  
        super.getDateCreated() + " and the radius is " + radius);  
}
```

โอเวอร์ไรด์เมธอด(Overriding Methods)

- ▶ Subclass จะสืบทอดข้อมูลจาก superclass บางครั้งที่ subclass ต้องการแก้ไขรายละเอียดการทำงานของเมธอดที่ประกาศไว้แล้วที่ superclass เราเรียกเทคนิคนี้ว่า Overriding Methods

```
public class Circle extends GeometricObject {  
    // Other methods are omitted  
  
    /** Override the toString method defined in GeometricObject */  
    public String toString() {  
        return super.toString() + "\nradius is " + radius;  
    }  
}
```

NOTE

- ❑ instance method สามารถโอเวอร์ไรต์ได้ถ้าประกาศแบบอนุญาตให้เข้าถึงได้ตั้งนั้น เมธอดที่มีการเข้าถึงแบบ private จะไม่สามารถ overridden
- ❑ ถ้าเมธอดที่ประกาศในคลาสลูกเป็นแบบ private เหมือนที่คลาสแม่ประกาศ กล่าวได้ว่าเมธอดสองตัวไม่เกี่ยวข้องกันโดยสิ้นเชิง
- ❑ instance method สามารถสืบทอดได้เหมือนกับ static method แต่ข้อแตกต่างคือ static method ไม่สามารถ overridden ถ้า static method ที่กำหนดใน superclass ถูกนิยามใหม่ใน subclass เมธอดใน superclass จะถูกซ่อน

Shadowing

- ▶ *Shadowing* refers to the practice in Java programming of using two variables with the same name within scopes that overlap.
- ▶ When you do that, the variable with the higher-level scope is hidden because the variable with lower-level scope overrides it.
- ▶ The higher-level variable is then “shadowed.”
- ▶ You can access a shadowed class or instance variable by fully qualifying it — that is, by providing the name of the class that contains it

Shadowing

```
► public class ShadowApp
{
    static int x;
    public static void main(String[] args)
    {
        x = 5;
        System.out.println("x = " + x);
        int x;
        x = 10;
        System.out.println("x = " + x);
        System.out.println("ShadowApp.x = " +
            ShadowApp.x);
    }
}
```

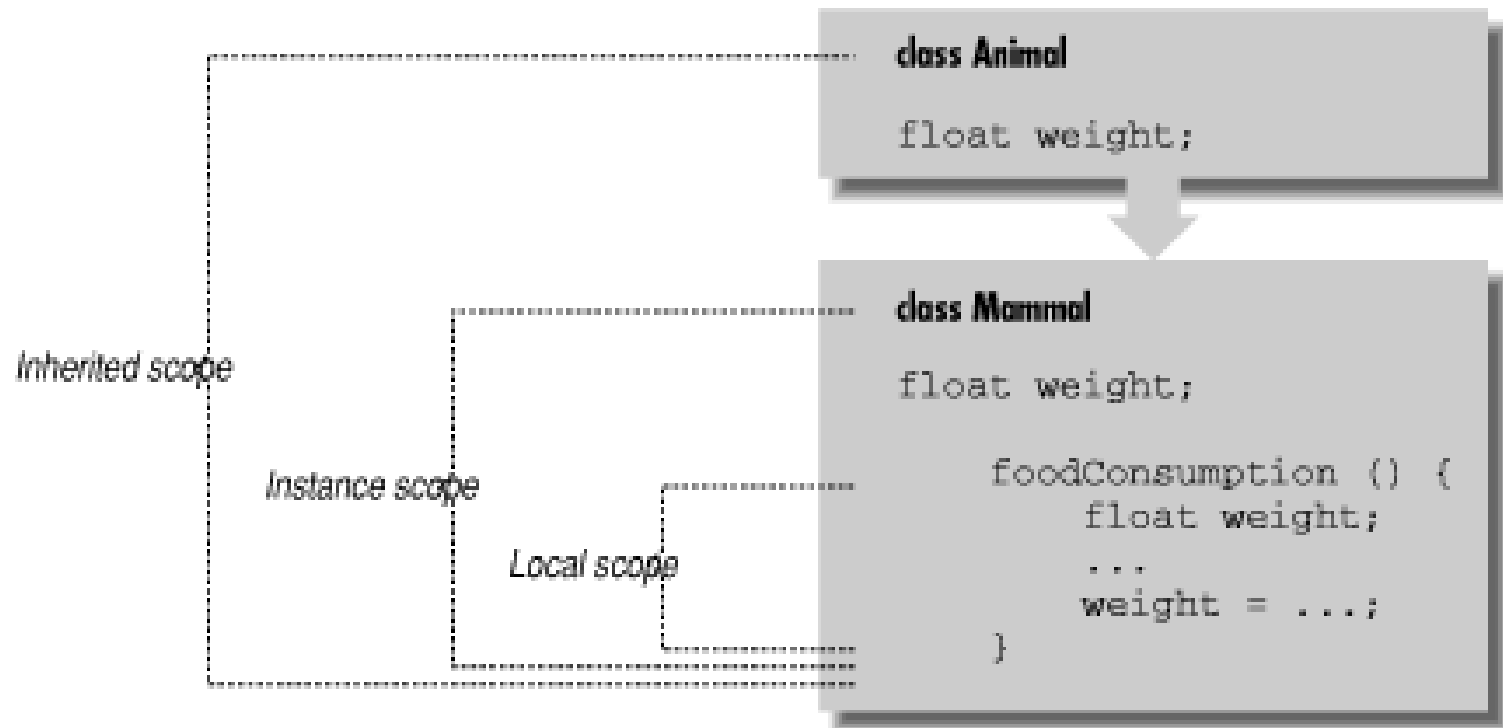
```
x = 5
x = 10
x = 10
ShadowApp.x = 5
```

Shadowing

```
▶ class Animal {  
    float weight;  
    ...  
    void eat() {  
        ...  
    }  
    ...  
}  
class Mammal extends Animal {  
    int heartRate;  
    // inherits weight  
    ...  
    void breathe() {  
        ...  
    }  
    // inherits eat()  
}  
class Cat extends Mammal {  
    boolean longHair;  
    // inherits weight and heartRate  
    ...  
    void purr() {  
        ...  
    }  
    // inherits eat() and breathe()  
}
```



Shadowing



Overriding vs. Overloading

```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overrides the method in B  
    public void p(double i) {  
        System.out.println(i);  
    }  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        A a = new A();  
        a.p(10);  
        a.p(10.0);  
    }  
}  
  
class B {  
    public void p(double i) {  
        System.out.println(i * 2);  
    }  
}  
  
class A extends B {  
    // This method overloads the method in B  
    public void p(int i) {  
        System.out.println(i);  
    }  
}
```


Static and Dynamic Binding

- Binding คือ การระบุว่า ชื่อ ที่ปรากฏในโปรแกรมแต่ละตำแหน่งนั้น หมายถึง ค่าใด
- หาก binding ใดเกิดขึ้นตั้งแต่ก่อนโปรแกรมเริ่มทำงาน และจะไม่เปลี่ยนแปลงระหว่างโปรแกรมทำงาน เราเรียกว่า static binding ซึ่งส่วนใหญ่เกิดขึ้นตอนคอมไพล์โปรแกรม และมีบางส่วนเกิดขึ้นตอนคลาสถูกโหลด
- หากเป็น binding ที่เกิดขึ้นระหว่างโปรแกรมทำงาน เราเรียกว่า dynamic binding ปกติจะเกิดขึ้นทุกครั้ง เมื่อโปรแกรมทำงานถึงตำแหน่งที่อ้างถึงชื่อนั้น

Static and Dynamic Binding

```
▶ // Binding.java
class A {

    int x = 1;
    String f() { return "A"; }

}
class B extends A {

    int x = 2;
    String f() { return "B"; }

}
class Binding {

    static void test(A a) {

        System.out.println(a.x +
a.f());

    }
    public static void main(String
args[]) {

        test(new A());

        test(new B());

    }

}
```

- ▶ ชื่อ data member (คือ x ในกรณีนี้) เมื่อถูกอ้างถึงโดย reference (คือ a) จะถูกทำ static binding ให้ a.x เป็น x ของคลาส A เพราะที่ก่อน value ของทั้งคลาส A และคลาส B จะมี x ของคลาส A อยู่ตำแหน่ง(ที่นับจาก byte แรกของก้อน value)เหมือนกัน ดังนั้น ไม่ว่า main() จะส่ง reference ของคลาส A หรือ B มาเป็นพารามิเตอร์ให้ test() ก็ตาม ค่า a.x ที่ถูกพิมพ์ออกมาจะเป็น x ของคลาส A ที่มีค่าเป็น 1
- ▶ แต่หากเป็นชื่อ method (คือ f() ในกรณีนี้) เมื่อถูกอ้างถึงโดย reference (คือ a ในกรณีนี้) จะถูกทำ dynamic binding คือ ทุกครั้งที่โปรแกรมทำงานถึงตำแหน่ง a.f() ค่า reference 'a' จะถูกใช้อ้างอิงไปที่ก่อน value ที่ชื่ออยู่ขณะนั้น ซึ่งจะมี reference ไปที่ class object แล้ว method 'f()' ใน class object นั้นจะถูกทำงาน ดังนั้นเมื่อ main() ส่ง reference ของคลาส A มาให้ x.f() ก็คือ f() ของคลาส A และหากส่ง reference ของคลาส B มาให้ x.f() ก็คือ f() ของคลาส B

The Object Class and Its Methods

- ▶ ทุก ๆ คลาสในภาษาจาวาจะถูกสืบทอดมาจากคลาส `java.lang.Object` ตัวอย่างคลาสที่สืบทอดมาจากคลาส `Object` เช่น `String`, `StringBuffer`, `StringTokenizer` ภายในคลาส `Object` จะมีเมธอดที่สำคัญที่มีการเรียกใช้งานบ่อย ๆ เช่น
 - ▶ `public Boolean equals(Object object)`
 - ▶ `public int hashCode`
 - ▶ `public String toString()`

```
public class Circle {  
    ...  
}
```

Equivalent

```
public class Circle extends Object {  
    ...  
}
```

ตัวอย่างการทำงานของเมธอด `toString()` ในคลาส `Object`

- ▶ เมธอด `toString()` จะคืนค่า `String` จากวัตถุ ค่าดีฟอลต์ที่คืนออกมาคือ string ที่อธิบายชื่อ class ที่วัตถุถูกสร้างขึ้นหรือคลาสที่วัตถุเป็น instance ตามด้วยเครื่องหมาย at sign (@) และตัวเลขที่แสดงวัตถุนั้น ๆ เช่น

```
Loan loan = new Loan();  
System.out.println(loan.toString());
```

ค่าที่คืนออกมาคือ `Loan@15037e5` จะพบว่าข้อมูลดังกล่าวไม่ได้นำไปใช้ประโยชน์เท่าที่ควร ดังนั้นควร override เมธอด `toString` เพื่อให้แสดงข้อมูลตามที่ต้องการ

The equals Method

- ▶ เมธอด equals() จะเปรียบเทียบข้อมูลภายในวัตถุ รายละเอียดของเมธอด equals ที่เขียนในคลาสObject คือ

```
public boolean equals(Object obj) {  
    return (this == obj);  
}
```

สามารถ overridden

เมธอด equals ใน
คลาส Circle ได้ดังนี้

```
public boolean equals(Object o) {  
    if (o instanceof Circle) {  
        return radius == ((Circle)o).radius;  
    }  
    else  
        return false;  
}
```

NOTE

- ▶ การเปรียบเทียบโดยใช้เครื่องหมาย `==` จะใช้เปรียบเทียบข้อมูลภายในชนิดข้อมูลแบบ primitive หรือตรวจสอบว่าวัตถุ 2 วัตถุมีการอ้างอิงที่ตำแหน่งเดียวกันหรือไม่ ส่วนการใช้เมธอด `equals` จะตรวจสอบว่าวัตถุ 2 วัตถุมีข้อมูลภายในเหมือนกันหรือไม่
- ▶ การเปรียบเทียบโดยการใช้เครื่องหมาย `==` จะมีน้ำหนักมากกว่าเมธอด `equals` เนื่องจากเครื่องหมาย `==` จะตรวจสอบว่าตัวแปรอ้างอิงวัตถุ 2 ตัวอ้างอิงไปยังวัตถุเดียวกันหรือไม่